

Guia de Aprendizaje 01

Audio Meter Studio - HTML, CSS, JavaScript y audio en tiempo real desde cero

Material de estudio para aprender programacion construyendo una app real. Version 1.0 - 26 de junio de 2026

Como usar esta guía

Esta guía no está pensada solo para explicar una app terminada. Está pensada para que puedas reconstruir mentalmente la app paso a paso y entender que hace cada pieza del código.

La idea es sencilla: primero construimos con ayuda de agentes en VS Code; después hacemos una disección pedagógica para convertir el proyecto en aprendizaje real. Aquí no buscamos memorizar sintaxis como un loro con teclado. Buscamos entender el circuito.

Objetivo final: que cuando veas una línea de JavaScript no sientas que estás mirando una partitura de free jazz escrita por una impresora enfadada.

Principio importante: no hace falta dominarlo todo hoy. Lo importante es reconocer patrones y repetirlos en apps reales.

1. Que hace Audio Meter Studio

Audio Meter Studio es una app web que usa el micrófono del navegador para analizar audio en tiempo real. Muestra el nivel de entrada, una estimación en dBFS, un peak hold, un indicador de clipping y una visualización en canvas que puede alternar entre waveform y spectrum.

- **P**ide permiso para usar el micrófono.
- **C**rea un entorno de audio con Web Audio API.
- **C**onecta el micrófono a un analizador.
- **L**ee datos de la señal de audio en cada frame.
- **C**alcula nivel RMS y dBFS aproximado.
- **A**ctualiza el medidor visual, el peak hold y el indicador de clipping.
- **D**ibuja la forma de onda o el espectro en un canvas.
- **P**ermite parar el micrófono y liberar la entrada de audio.

Aviso honesto: esto no es todavía un medidor profesional calibrado de mastering. Es una herramienta web educativa y funcional. Para medición profesional real harían falta calibración, control de interfaz, niveles de referencia y validación técnica adicional.

2. Mapa del proyecto

Estructura recomendada

```
audio-meter-studio/  
├── index.html  -> estructura visual, botones, medidor y canvas  
├── app.js      -> logica de micrófono, analisis, dibujo y estado  
└── README.md  -> descripción del proyecto para GitHub
```

El HTML define lo que existe en pantalla. El CSS define el aspecto. El JavaScript define el comportamiento. Dicho en modo estudio: HTML es el escenario, CSS es la iluminación y JS es el grupo tocando.

3. Conceptos que aprendes con esta app

Concepto	Explicación principiante	Donde aparece en la app
HTML	Crea la estructura de la página.	Botones, medidor, estadísticas, canvas.

CSS	Da estilo visual a la interfaz.	Colores, tarjeta central, barras, indicador de clipping.
JavaScript	Hace que la app reaccione y procese audio.	Eventos, microfono, canvas, calculos.
DOM	Permite que JS encuentre y modifique elementos del HTML.	document.getElementById(...).
Eventos	Acciones que ocurren y disparan código.	Click en Start, Stop, Fullscreen y Switch.
async/await	Forma clara de esperar operaciones que tardan.	Permiso de microfono.
getUserMedia	Pide acceso al microfono.	navigator.mediaDevices.getUserMedia.
AudioContext	Crea el sistema de audio del navegador.	new AudioContext().
AnalyserNode	Extrae datos analizables de la señal.	audioContext.createAnalyser().
RMS	Mide energía media de la señal.	Calculo con sumSquares.
dBFS	Nivel respecto al máximo digital.	$20 * \text{Math.log10}(\text{rms})$.
Canvas	Lienzo para dibujar en tiempo real.	Waveform y spectrum.
requestAnimationFrame	Actualiza la pantalla al ritmo del navegador.	updateMeter en bucle.
Estado	Variables que recuerdan como esta la app.	isListening, viewMode, peakHold.

4. Construcción paso a paso

Paso 1 - Crear los elementos basicos en HTML

Empezamos con lo minimo: un titulo, un boton y un texto de estado.

HTML minimo

```
<h1>Audio Meter Studio</h1>
<button id="startBtn">Start microphone</button>
<p id="statusText">Idle</p>
```

El atributo id es fundamental: permite que JavaScript encuentre ese elemento concreto. Sin id, JS va con linterna por una nave industrial.

Paso 2 - Conectar HTML con JavaScript

Primer evento

```
const startBtn = document.getElementById('startBtn');
const statusText = document.getElementById('statusText');

startBtn.addEventListener('click', () => {
  statusText.textContent = 'Button clicked';
});
```

Aqui aparecen tres ideas clave: guardamos referencias a elementos, escuchamos un click y cambiamos texto en pantalla.

Parte	Significado
const startBtn	Crea una constante llamada startBtn.
document.getElementById('startBtn')	Busca el elemento del HTML con ese id.
addEventListener('click', ...)	Ejecuta codigo cuando ocurre un click.
textContent	Cambia el texto visible de un elemento.

Paso 3 - Declarar el estado de la app

Una app necesita recordar cosas. Por ejemplo: si esta escuchando, que vista esta activa, cual fue el ultimo pico, etc.

Variables de estado

```
let audioContext;
let analyser;
let animationFrame;
let source;
let stream;
let peakHold = 0;
let peakHoldReleaseTime = 0;
let lastFrameTime = 0;
let viewMode = 'waveform';
let isListening = false;
```

Regla sencilla: usa const cuando el valor no se reasigna; usa let cuando el valor puede cambiar durante la vida de la app.

Paso 4 - Preparar el canvas

El canvas es un lienzo. En esta app sirve para dibujar la forma de onda o el espectro. La funcion resizeCanvas ajusta el lienzo a pantallas normales y retina.

Canvas preparado para buena resolucion

```
const resizeCanvas = () => {
  const ratio = window.devicePixelRatio || 1;
  const width = canvas.clientWidth;
  const height = canvas.clientHeight;
  canvas.width = width * ratio;
  canvas.height = height * ratio;
  ctx.setTransform(ratio, 0, 0, ratio, 0, 0);
};
```

Paso 5 - Pedir acceso al microfono

El navegador no permite usar el microfono sin permiso. Por eso esta parte debe ejecutarse tras una accion del usuario, normalmente un click.

Entrada de audio sin procesamiento automatico

```
stream = await navigator.mediaDevices.getUserMedia({
  audio: {
    echoCancellation: false,
    noiseSuppression: false,
    autoGainControl: false,
  },
});
```

Las tres opciones en false intentan evitar que el navegador cambie la senal con cancelacion de eco, reduccion de ruido o ganancia automatica. No convierte el navegador en un Apogee de 3.000 euros, pero evita maquillaje innecesario.

Paso 6 - Crear AudioContext y AnalyserNode

Sistema de audio

```
audioContext = new AudioContext();
analyser = audioContext.createAnalyser();
analyser.fftSize = 2048;
analyser.smoothingTimeConstant = 0.9;
```

Piensa en AudioContext como una mesa de mezclas dentro del navegador. El analyser es como insertar un analizador en la cadena para observar la senal.

fftSize define cuantas muestras se usan para el analisis temporal. smoothingTimeConstant suaviza la lectura del analizador de frecuencia.

Paso 7 - Conectar microfono -> analizador

Cadena de audio

```
source = audioContext.createMediaStreamSource(stream);
source.connect(analyser);
```

Mapa mental

Microfono -> MediaStreamSource -> AnalyserNode -> datos para la interfaz

Paso 8 - Leer datos de audio

Datos de tiempo y frecuencia

```
const timeData = new Uint8Array(analyser.fftSize);
const frequencyData = new Uint8Array(analyser.frequencyBinCount);
analyser.getBytesTimeDomainData(timeData);
analyser.getBytesFrequencyData(frequencyData);
```

timeData representa la forma de onda. frequencyData representa energia por bandas de frecuencia. Si la app fuera un estudio, timeData seria mirar la onda en el editor, frequencyData seria mirar un analizador de espectro.

Paso 9 - Calcular RMS

Calculo RMS

```
let sumSquares = 0;
for (let i = 0; i < timeData.length; i += 1) {
  const sample = (timeData[i] - 128) / 128;
  sumSquares += sample * sample;
}
const rms = Math.sqrt(sumSquares / timeData.length);
```

RMS significa Root Mean Square. En lenguaje humano: una forma de medir la energia media de la senal. No se queda solo con un pico aislado; mira la fuerza general.

Analogicamente: un golpe de caja puede pegar un pico fuerte, pero un bajo sostenido puede tener mas energia media. RMS ayuda a entender esa energia continua.

Paso 10 - Convertir RMS a dBFS aproximado

dBFS

```
const dbFs = rms > 0 ? 20 * Math.log10(rms) : -Infinity;
```

dBFS significa decibelios respecto al maximo digital. En digital, 0 dBFS es el techo. Valores por debajo son negativos. Silencio teorico seria -infinito.

Nivel	Interpretacion rapida
0 dBFS	Techo digital. Riesgo claro de clipping.
-1 dBFS	Muy cerca del limite.
-6 dBFS	Fuerte, zona caliente.
-18 dBFS	Zona de trabajo comoda en muchos contextos.
-infinito	Silencio o senal inexistente.

Paso 11 - Actualizar la interfaz

Del calculo a la pantalla

```
fill.style.width = `${level}%`;
peak.style.left = `${peakHold}%`;
levelValue.textContent = `${level.toFixed(0)}%`;
peakValue.textContent = `${peakHold.toFixed(0)}%`;
dbValue.textContent = `${Number.isFinite(dbFs) ? dbFs.toFixed(1) : '-∞'} dBFS`;
```

Aqui JavaScript convierte calculos en cambios visibles: anchura de barra, posicion del pico, textos numericos y estado.

Paso 12 - Peak hold

Peak hold con retencion temporal

```
if (level > peakHold) {
  peakHold = level;
  peakHoldReleaseTime = timestamp + 900;
} else if (timestamp >= peakHoldReleaseTime) {
  peakHold = Math.max(0, peakHold - 40 * deltaSeconds);
}
```

El peak hold no baja inmediatamente. Primero se queda retenido unos milisegundos y despues cae progresivamente. Esto ayuda a leer picos sin que la vista vaya corriendo detras de la barrita como gato detras de laser.

Paso 13 - Indicador de clipping

Estados de nivel

```

if (dbFs >= 0) {
  state = 'clipping';
  label = 'Clipping';
} else if (dbFs >= -1) {
  state = 'hot';
  label = 'Clipping risk';
} else if (dbFs >= -6) {
  state = 'active';
  label = 'Hot';
}

```

La app transforma rangos de dBFS en etiquetas comprensibles. Esto es UX aplicada al audio: no solo mostrar numeros, sino explicar que significan.

Paso 14 - Dibujar waveform o spectrum

La funcion drawVisualizer decide que dibujar segun viewMode. Si viewMode es spectrum, usa frequencyData para barras. Si no, usa timeData para dibujar la onda.

Selector de vista

```

if (viewMode === 'spectrum') {
  // draw frequency bars
} else {
  // draw waveform
}

```

Este patron se repite muchisimo en programacion: una variable de estado decide que rama del codigo se ejecuta.

Paso 15 - Bucle de actualizacion**Animacion en tiempo real**

```
animationFrame = window.requestAnimationFrame(updateMeter);
```

requestAnimationFrame pide al navegador que vuelva a ejecutar updateMeter en el proximo frame. Asi la app se actualiza continuamente mientras escucha el microfono.

Paso 16 - Parar el microfono**Cerrar correctamente**

```

const stopMicrophone = () => {
  cancelAnimationFrame(animationFrame);

  if (source) {
    source.disconnect();
    source = null;
  }

  if (stream) {
    stream.getTracks().forEach((track) => track.stop());
    stream = null;
  }

  isListening = false;
  resetMeterState();
};

```

Esta funcion es importante porque una app responsable debe liberar el microfono. No basta con esconder el medidor: hay que parar los tracks y desconectar la fuente.

5. Clinica de sintaxis para esta app

Esta seccion resume la sintaxis que mas aparece. No hace falta memorizarla perfecta de golpe. La vas a reconocer por repeticion.

Sintaxis	Que significa	Ejemplo
const	Variable que no se reasigna.	const canvas = ...
let	Variable que puede cambiar.	let peakHold = 0;
() => {}	Funcion flecha.	const reset = () => { ... };
if / else	Condiciones.	if (dbFs >= -1) { ... }
for	Bucle.	for (let i = 0; i < timeData.length; i += 1)
await	Esperar una promesa.	await getUserMedia(...).
try/catch	Intentar algo y capturar errores.	try { ... } catch (error) { ... }
template string	Texto con variables dentro.	`\${level}%`
?.	Optional chaining: si existe, llama.	requestFullscreen?.()
===	Comparacion estricta.	viewMode === 'spectrum'
Math.max	Devuelve el mayor valor.	Math.max(0, peakHold - x)
Array / Uint8Array	Lista numerica.	new Uint8Array(...)

6. Ejercicios recomendados

Nivel 1 - Calentamiento

- 1 Cambia el texto del boton Start microphone por Start audio input.
- 2 Cambia el titulo de la pagina a Vitalab Audio Meter Studio.
- 3 Cambia el texto de privacidad manteniendo el mismo significado.
- 4 Cambia el umbral de Hot de -6 dBFS a -9 dBFS.
- 5 Haz que el peak hold se mantenga 1200 ms en vez de 900 ms.

Nivel 2 - Comprension

- 1 Explica con tus palabras que hace source.connect(analyser).
- 2 Explica por que hay que detener stream.getTracks().
- 3 Localiza donde se calcula RMS y comenta cada linea.
- 4 Localiza donde se cambia entre waveform y spectrum.
- 5 Añade un texto RMS en pantalla sin borrar el valor dBFS.

Nivel 3 - Mini mejoras

- 1 Añade un boton Reset peak hold.
- 2 Añade un modo Minimal view que oculte el canvas.
- 3 Añade una pequena leyenda: Safe / Hot / Clipping risk.
- 4 Haz que el indicador de clipping parpadee cuando llegue a Clipping.
- 5 Guarda el nivel maximo alcanzado durante la sesion.

7. Plantilla para repetir este metodo con cada app

Si creas un proyecto de aprendizaje, cada app puede convertirse automaticamente en una unidad de estudio. La estructura recomendada seria:

Proyecto maestro de aprendizaje

```
vitalab-learning-lab/
├── apps/
│   └── 01-audio-meter-studio/
│       ├── index.html
│       ├── app.js
│       └── README.md
├── guides/
│   └── 01-audio-meter-studio-guide.pdf
├── questionnaires/
│   └── 01-audio-meter-studio-quiz.pdf
├── syntax-clinic/
│   └── javascript-patterns.md
├── prompts/
│   └── app-review-template.md
└── progress-log.md
```

Flujo recomendado para cada app:

- 1 Construir una primera version con VS Code y agente.
- 2 Probarla manualmente.
- 3 Pasarme index.html, app.js y otros archivos relevantes.
- 4 Hacer revision tecnica: bugs, estructura, UX, seguridad y mejoras.
- 5 Implementar mejoras.
- 6 Crear guia de aprendizaje desde cero.
- 7 Crear cuestionario para responder sin mirar.
- 8 Anotar dudas de sintaxis y trabajarlas con ejercicios.

Este metodo convierte cada app en tres cosas a la vez: proyecto real, clase practica y prueba de progreso.

8. Mini glosario

Termino	Definicion sencilla
DOM	Representacion del HTML que JavaScript puede leer y modificar.
Evento	Algo que ocurre: click, resize, fullscreenchange.
Stream	Flujo de datos del microfono.
Track	Pista concreta dentro del stream, por ejemplo audio.
AudioContext	Entorno de audio del navegador.
AnalyserNode	Nodo que permite leer datos de la senal.
RMS	Medida de energia media de la senal.
dBFS	Nivel digital relativo al maximo posible.
Canvas	Zona donde JS dibuja graficos.
Waveform	Forma de onda en el tiempo.

Spectrum	Energia por frecuencias.
Clipping	Distorsion por superar el techo digital.
Refactor	Reorganizar codigo sin cambiar lo que hace.

Cierre

Esta primera app es pequena, pero contiene conceptos muy potentes: entrada de microfono, Web Audio API, canvas, bucle de animacion, calculo de RMS, dBFS aproximado, estado de interfaz y manejo de errores.

La sintaxis se ira asentando con repeticion. Lo importante es que ya estas aprendiendo con proyectos que tienen sentido para ti. Eso multiplica la memoria y la motivacion.